Csharp_week4_Day_3_Hands_on_Gurunath_Rageni

Level-1 Problem 1: Student Grade Calculator

Scenario:

A school wants to calculate the average marks of a student using a class-based approach.

Requirements:

1. Create a class Student.

2. Create method CalculateAverage(int m1, int m2, int m3).

3. Return the average marks.

4. Display grade based on average.

Technical Constraints:

1. Use return type double for average.

2. Avoid hard-coded values.

Expectations:

Clear separation of logic inside methods.

Learning Outcome:

Learn method creation, return values, and basic OOP concepts.

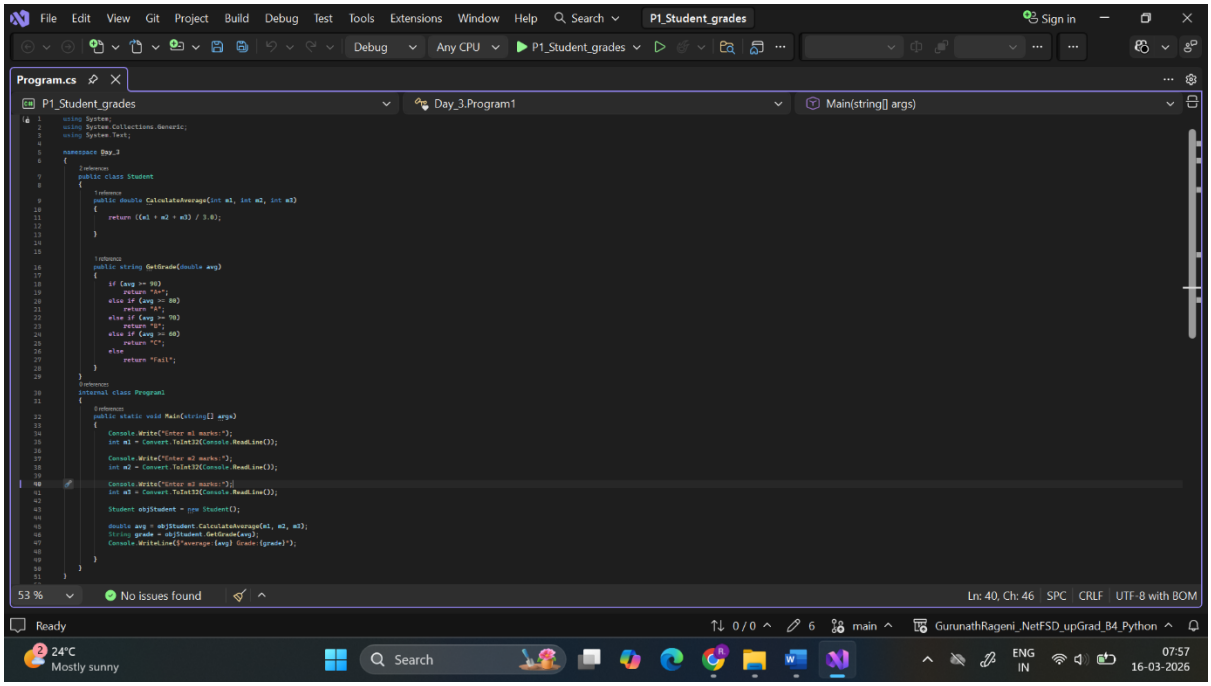
Sample Input:

80 70 90

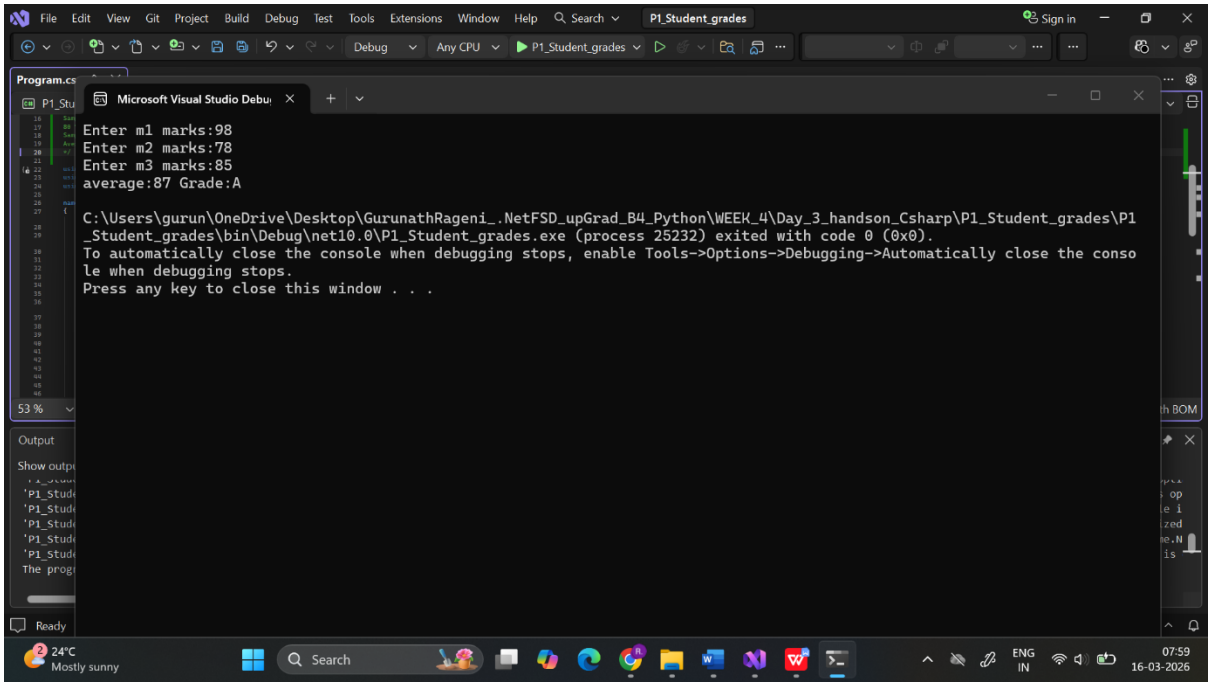
Sample Output:

Average = 80, Grade = A

Code screenshot:



Output:



Level-2 Problem 1: Bank Account with Encapsulation

Scenario:

A bank wants to manage customer accounts securely using encapsulation.

Requirements:

1. Create class BankAccount.

2. Private field: balance.

3. Public methods: Deposit(double amount), Withdraw(double amount).

4. Method GetBalance() to return balance.

5. Prevent withdrawal if insufficient balance.

Technical Constraints:

1. Balance must be private.

2. Access balance only through public methods.

3. Use appropriate return types.

Expectations:

Proper use of encapsulation and object-oriented principles.

Learning Outcome:

Understand encapsulation, access modifiers, and secure data handling.

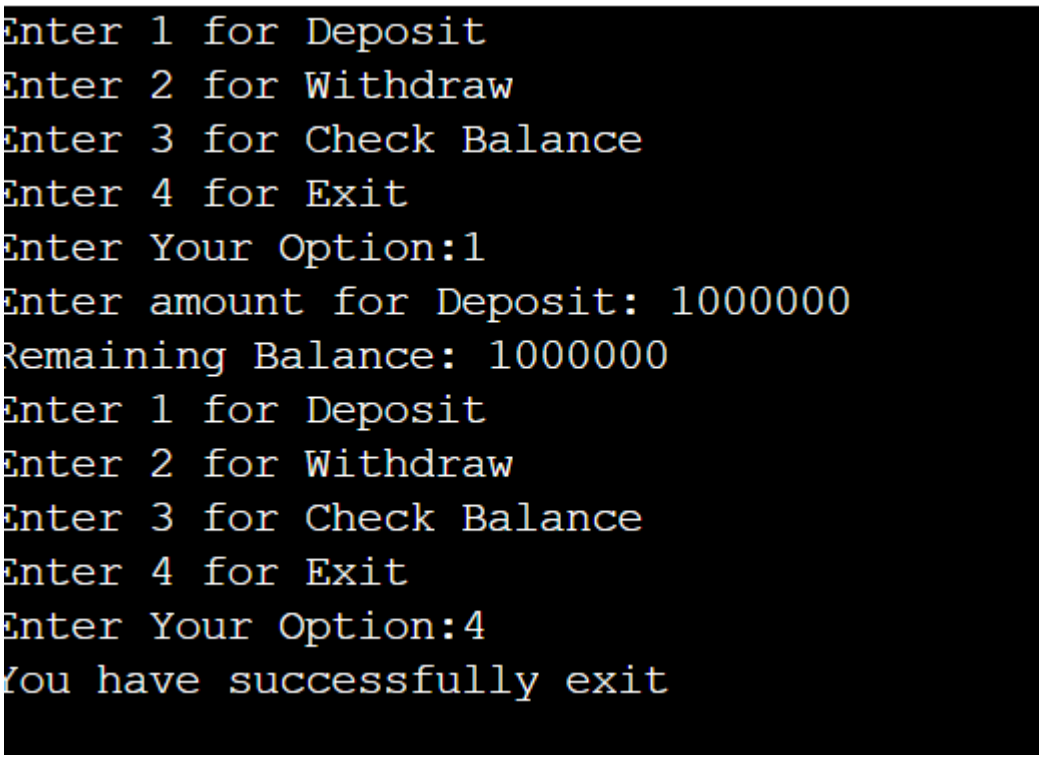
Sample Input:

Deposit 1000, Withdraw 300

Sample Output:

Current Balance = 700

Code screenshot:



Level-2 Problem 2: Bank Account Management System

Scenario:

A bank wants to develop a simple console-based application to manage customer bank accounts. The system should protect account balance information and allow controlled access using properties.

Requirements:

1. Create a BankAccount class with private fields for account number and balance.

2. Use properties to allow controlled access to account number and balance.

3. Implement Deposit and Withdraw methods with proper validation.

4. Prevent withdrawal if balance is insufficient.

Technical Constraints:

• Use private fields with public properties.

• Apply encapsulation and data hiding.

• No direct access to balance field from outside the class.

Expectations:

• Demonstrate correct use of access modifiers.

• Validate negative deposit or withdrawal amounts.

• Display updated balance after each transaction.

Learning Outcome:

• Understand encapsulation using properties.

• Apply data hiding effectively.

• Implement validation logic inside class methods.

Sample Input:

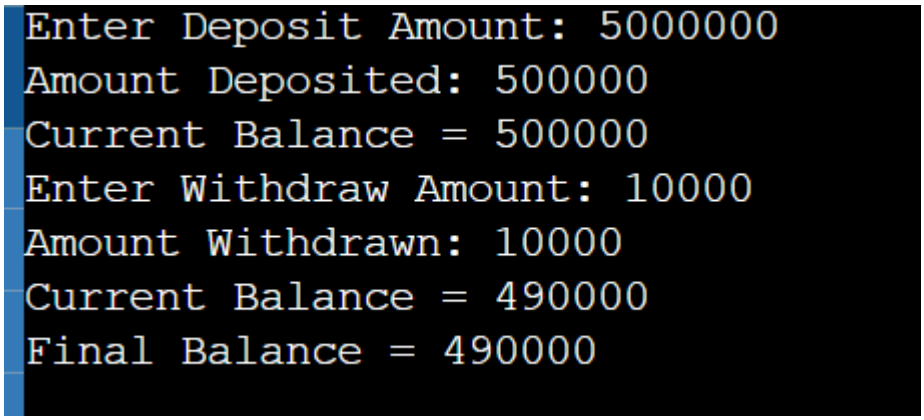
Deposit = 5000, Withdraw = 2000

Sample Output:

Current Balance = 3000

Code screenshot:

Output Screenshot:



Level-2 Problem 3: Employee Salary Calculator

Scenario:

A company wants to calculate employee salaries using inheritance. All employees have a base salary, but different roles calculate bonuses differently.

Requirements:

1. Create a base class Employee with Name and BaseSalary properties.

2. Create derived classes Manager and Developer.

3. Override a virtual method CalculateSalary().

4. Manager gets 20% bonus, Developer gets 10% bonus.

Technical Constraints:

• Use inheritance and method overriding.

• Apply polymorphism using base class reference.

• Use properties for salary details.

Expectations:

• Demonstrate runtime polymorphism.

• Avoid code duplication.

• Display final calculated salary.

Learning Outcome:

• Understand inheritance hierarchy.

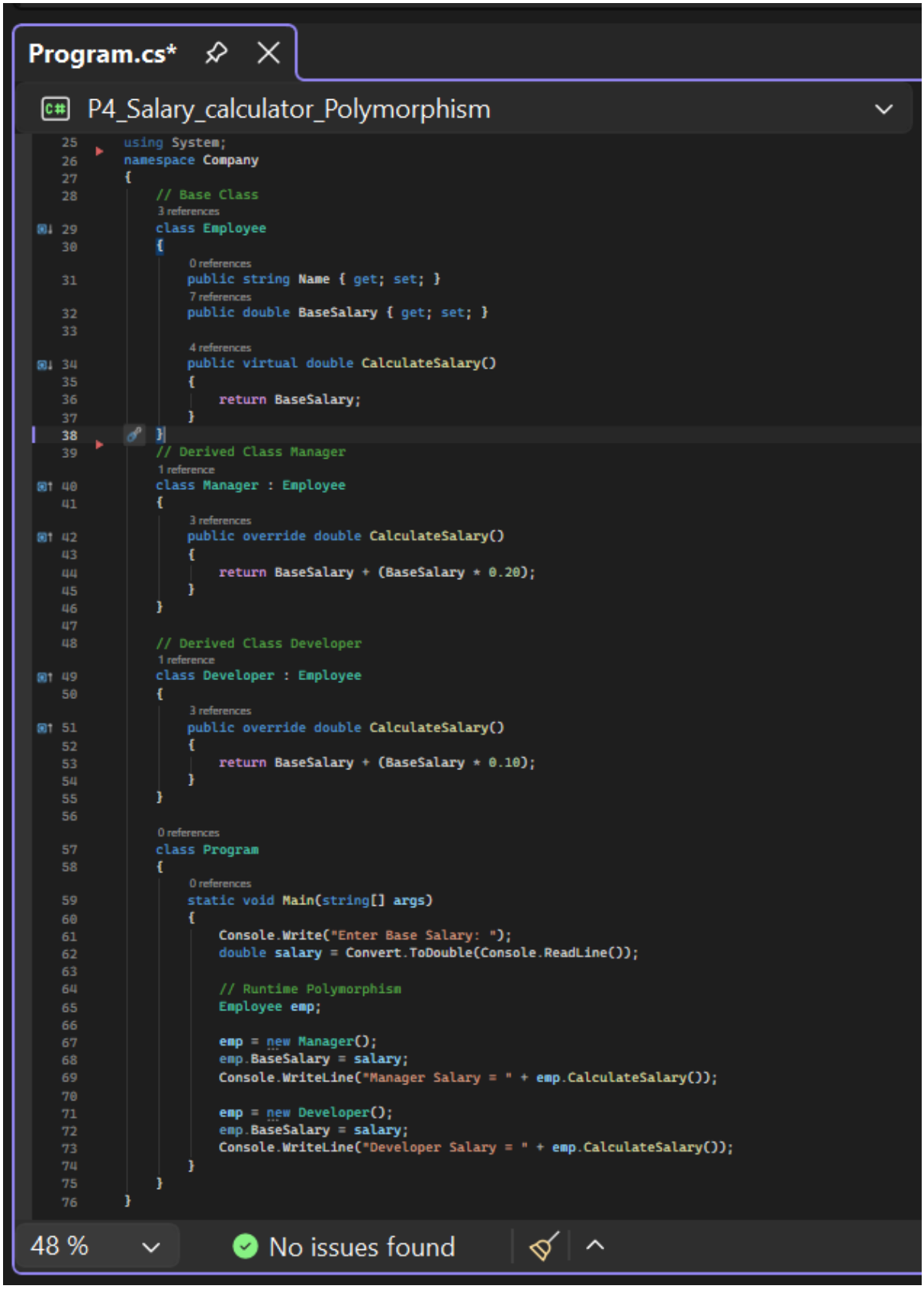
Sample Input:

BaseSalary = 50000

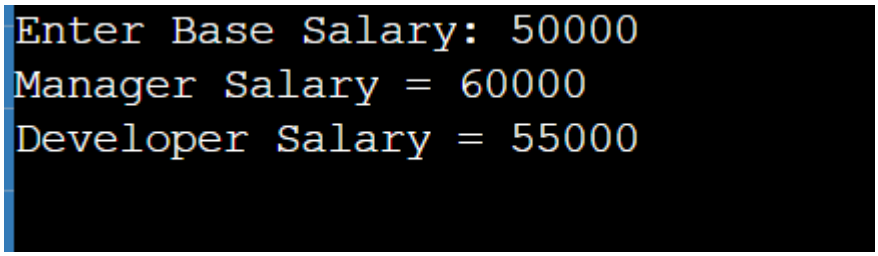
Sample Output:

Manager Salary = 60000, Developer Salary = 55000

Code Screenshot:



Output Screenshot:



Level-2 Problem 4: Online Shopping Cart System

Scenario:

An e-commerce platform needs a flexible cart system where different product types calculate discounts differently.

Requirements:

1. Create a base class Product with properties Name and Price.

2. Create derived classes Electronics and Clothing.

3. Implement a virtual method CalculateDiscount().

4. Electronics get 5% discount, Clothing gets 15% discount.

5. Use encapsulation to protect price updates.

Technical Constraints:

• Use private fields with public properties.

• Apply inheritance and method overriding.

• Prevent negative price assignment.

Expectations:

• Demonstrate polymorphic behavior in cart processing.

• Apply data validation inside properties.

• Calculate and display final price after discount.

Learning Outcome:

• Combine encapsulation and polymorphism.

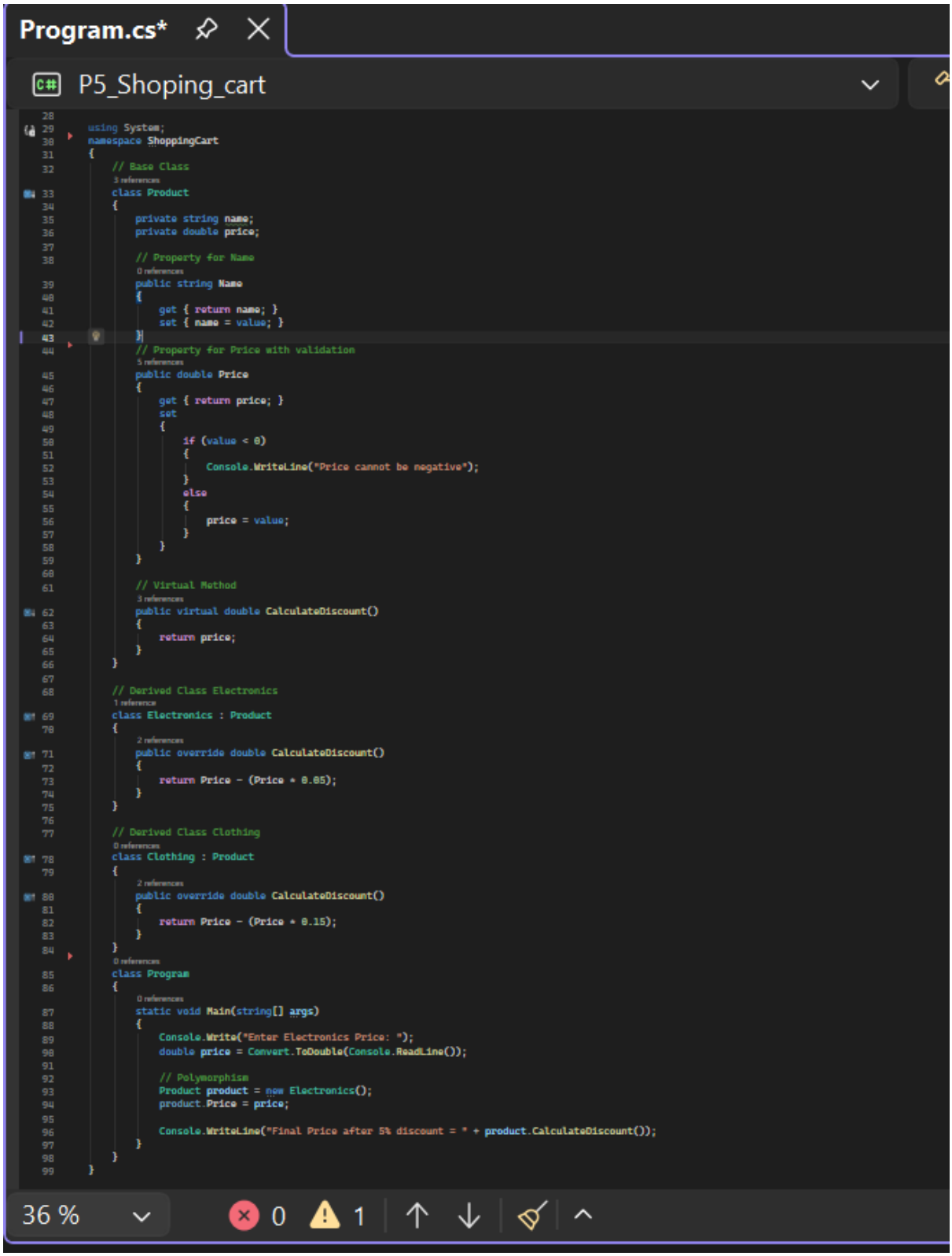
• Design extensible product hierarchy.

• Implement business logic in overridden methods.

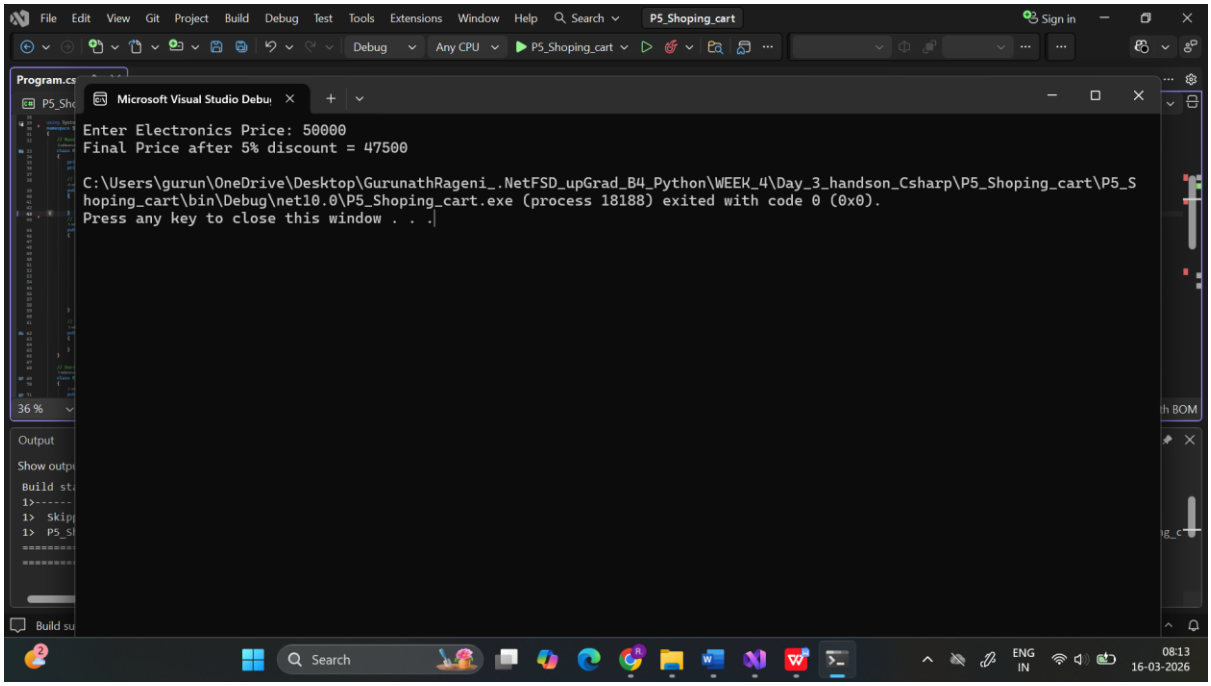
Sample Input: Electronics Price = 20000

Sample Output: Final Price after 5% discount = 19000

Code Screenshot:



Output Screenshot:



Level-2 Problem 5: Vehicle Rental System

Scenario:

A vehicle rental company wants a system where different vehicle types calculate rental charges differently.

Requirements:

1. Create a base class Vehicle with properties Brand and RentalRatePerDay.

2. Create derived classes Car and Bike.

3. Override CalculateRental(int days) method.

4. Car adds insurance charge of 500 per rental.

5. Bike offers 5% discount on total rental.

Technical Constraints:

• Use encapsulation with proper access modifiers.

• Apply runtime polymorphism.

• Validate number of rental days.

Expectations:

• Use base class reference to call overridden methods.

• Implement clean class hierarchy.

• Display final rental cost.

Learning Outcome:

• Master inheritance and polymorphism.

• Implement real-world OOP scenarios.

• Improve object-oriented design skills.

Sample Input:

Car RentalRatePerDay = 2000, Days = 3

Sample Output:

Total Rental = 6500

Code Screenshot:

Output Screenshot: